

SLATE

User Manual

Strategy Learning & Autonomous Trading Engine

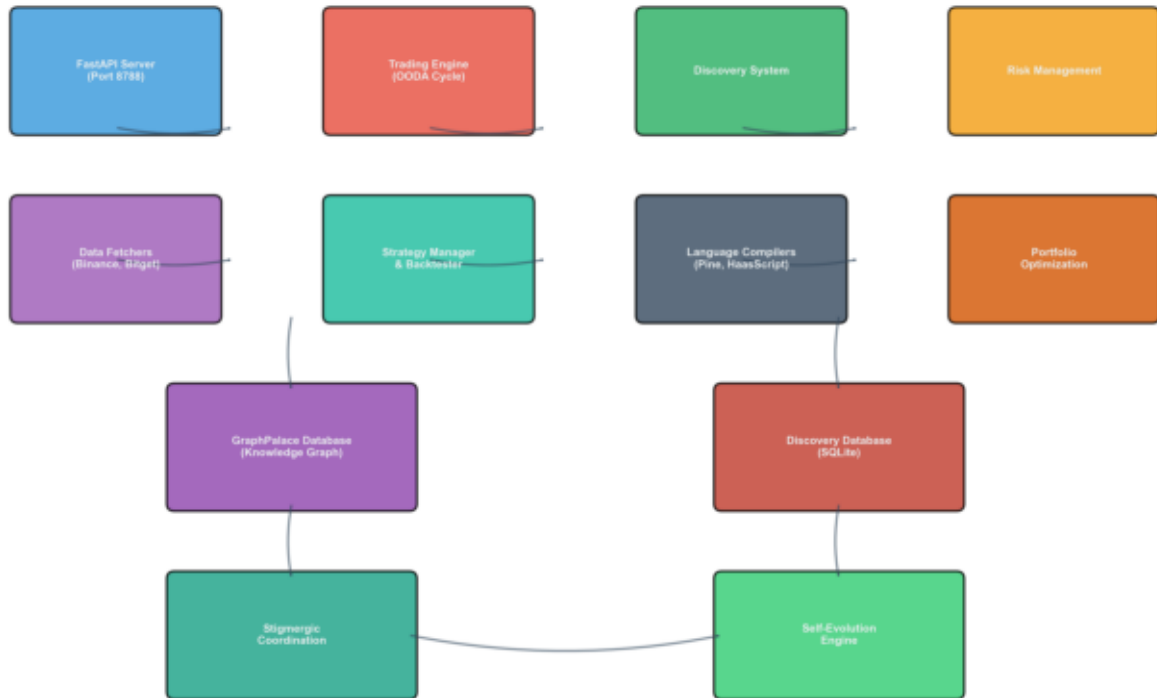
Version 1.0.0

April 30, 2026

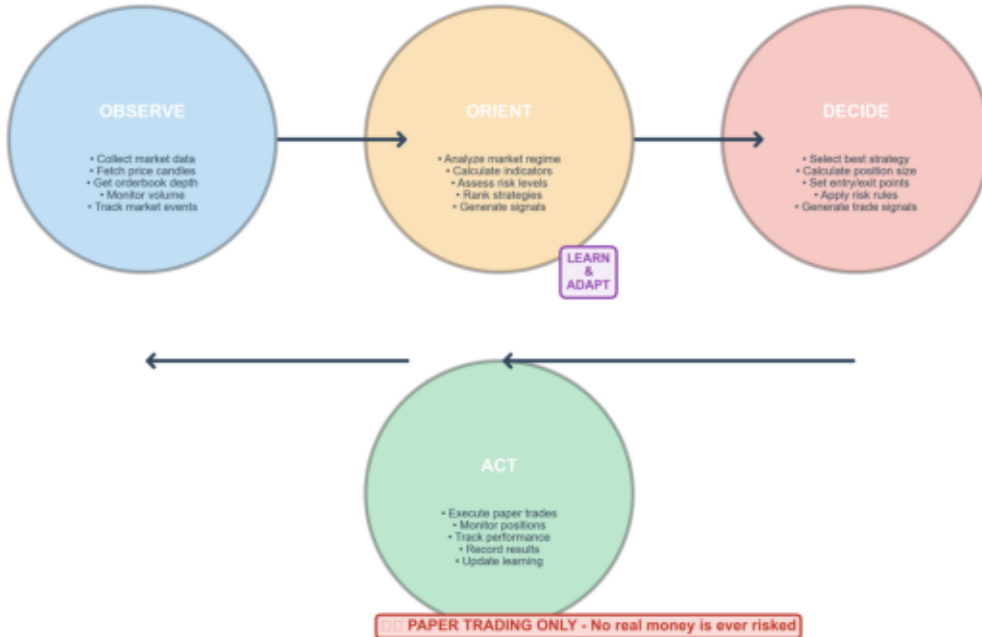
 **PAPER TRADING ONLY**

SLATE User Manual

SLATE Architecture Overview



SLATE OODA Trading Cycle



SLATE Discovery System



****Key Features:****

- ****Realistic Backtesting****: 0.02% maker fee, 0.05% taker fee, 5 basis points slippage
- ****Multi-Path Testing****: Tests each strategy across 100+ bootstrapped price paths
- ****Stigmergic Coordination****: Swarm intelligence ensures strategy diversity
- ****Self-Evolution****: Best strategies are automatically optimized over time

Risk Management

- ****Kelly Criterion****: Calculates optimal position sizes
- ****Value at Risk (VaR)****: Measures potential losses
- ****Drawdown Control****: Limits maximum drawdown
- ****Portfolio Optimization****: Balances risk across strategies

Data Management

- ****GraphPalace Database****: Knowledge graph storing strategy relationships
- ****Discovery Database****: SQLite database with tiered storage
- ****Historical Data Archive****: Cached market data for backtesting

2. Installation Guide

System Requirements

****Operating System:****

- macOS 10.15+ (Catalina or later)
- Linux (Ubuntu 20.04+, Debian 11+, or similar)
- Windows 10/11 with WSL2

****Hardware:****

- Minimum: 4GB RAM, 2 CPU cores
- Recommended: 8GB+ RAM, 4+ CPU cores
- Storage: 500MB free space (+ additional for historical data)

****Software:****

- Python 3.8 or higher
- pip (Python package installer)

Step-by-Step Installation

Step 1: Install Python

****On macOS:****

Install Homebrew if not already installed

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```


Install Python

```
brew install python@3.11
```

```
**On Linux (Ubuntu/Debian):**
```

```
sudo apt update
```

```
sudo apt install python3.11 python3-pip python3-venv
```

```
**On Windows:**
```

```
Download and install Python from [python.org](https://www.python.org/downloads/)
```

```
#### Step 2: Download SLATE
```

Clone from GitHub

```
git clone https://github.com/Tilanthi/SLATE.git
cd SLATE
Or download the ZIP file from GitHub and extract it.
#### Step 3: Install Dependencies
SLATE requires several Python packages. Install them with:
pip install -r requirements.txt
**Required packages:**
- `fastapi` - Web framework for the API server
- `uvicorn` - ASGI server for running FastAPI
- `numpy`, `pandas` - Data processing
- `scipy` - Statistical calculations
- `aiohttp` - Async HTTP client
- `ccxt` - Cryptocurrency exchange library
- `pytest` - Testing framework
#### Step 4: Verify Installation
Run the test suite to verify everything works:
python3 slate_core/run_tests.py
You should see:
Results: 31 passed, 0 failed
#### Step 5: Configure Environment (Optional)
Create a `.env` file in the SLATE directory:
cp .env.example .env
Edit `.env` to customize settings:
```

Server Configuration

```
SLATE_PORT=8788      # Default port for the web interface
SLATE_HOST=0.0.0.0   # Listen on all interfaces
```

GraphPalace (Optional)

GRAPHPALACE_ENABLED=false # Enable advanced knowledge graph
GRAPHPALACE_PATH=./slate_core/palace_data

4. Starting SLATE

Start the SLATE server with:
python3 -m slate_core.server
You should see output like:
INFO: Started server process [12345]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8788
Accessing the Dashboard
Once SLATE is running, open your web browser and navigate to:
Main Dashboard: http://localhost:8788/dashboard
This shows:
- Active strategies
- Current positions (paper trading)
- Performance metrics
- System status
Discovery Dashboard: http://localhost:8788/discovery-dashboard
This shows:
- Real-time discovery statistics
- Top performing strategies
- Evolution progress
- Performance charts
API Documentation: http://localhost:8788/docs
Interactive API documentation with testing capabilities.
Quick Test
Let's run a quick discovery cycle to verify everything works:
1. Open a new terminal window
2. Run this command:
curl -X POST http://localhost:8788/api/discovery/trigger
3. Wait a few seconds, then check the Discovery Dashboard
You should see new strategies being discovered and tested!

5. The Dashboard Interface

The dashboard includes dashboards for monitoring and control:
Main Dashboard
Located at `http://localhost:8788/dashboard`
Features:
Left Panel - Strategy Management:
- Create new strategies
- Activate/deactivate strategies
- View strategy performance
- Edit strategy parameters
Center Panel - Performance:
- Real-time P&L chart
- Win/Loss ratio
- Sharpe ratio
- Maximum drawdown
Right Panel - Positions:
- Current open positions
- Position sizes
- Entry/exit prices
- Unrealized P&L
Bottom Panel - Activity Feed:
- Recent trades
- System events
- Discovery notifications
- Error messages
Discovery Dashboard
Located at `http://localhost:8788/discovery-dashboard`
Features:
Statistics Cards:
- Total tests run
- Best Sharpe ratio
- Best return percentage
- Profitable strategy percentage
Top Strategies Table:
- Strategy name
- Return percentage
- Sharpe ratio
- Max drawdown
- Win rate
Performance Charts:
- Return distribution
- Sharpe ratio over time
- Strategy type breakdown
Control Buttons:
- Start/Stop discovery
- Trigger single cycle
- Cleanup database
- Export results
Navigation
Both dashboards are accessible from the top navigation bar. You can switch between them seamlessly while SLATE continues running in the background.

6. Creating Trading Strategies

SLATE includes 10+ built-in strategy types:
1. **Momentum**: Trades based on price momentum
2. **Mean Reversion**: Trades price reversals using Bollinger Bands
3. **Breakout**: Trades breakouts from consolidation
4. **Trend Following**: Uses moving average crossovers
5. **Statistical Arbitrage**: Z-score based pairs trading
6. **Machine Learning**: Uses ML for signal generation
7. **Regime Switching**: Adapts to market conditions
8. **Order Flow**: Trades based on orderbook imbalances
9. **Microstructure**: Exploits market inefficiencies
10. **Multi-Timeframe**: Combines signals across timeframes
Creating Strategies via API
Example 1: Create a Simple Momentum Strategy
Send a POST request to create a strategy:
POST http://localhost:8788/api/strategies
{
 "name": "BTC Momentum 1H",
 "type": "momentum",
 "symbol": "BTCUSDT",
 "timeframe": "1h",
 "parameters": {
 "period": 14,
 "threshold": 0.02
 }
}
Example 2: Create a Mean Reversion Strategy
POST http://localhost:8788/api/strategies
{
 "name": "ETH Bollinger Bands",
 "type": "mean_reversion",
 "symbol": "ETHUSDT",
 "timeframe": "15m",
 "parameters": {
 "period": 20,
 "std_dev": 2.0,
 "entry_threshold": 0.95
 }
}
Example 3: Create a Trend Following Strategy
POST http://localhost:8788/api/strategies
{
 "name": "BTC Trend Follower",
 "type": "trend_following",
 "symbol": "BTCUSDT",
 "timeframe": "4h",
 "parameters": {
 "fast_period": 12,
 "slow_period": 26,
 "signal_period": 9
 }
}
Strategy Parameters
Each strategy type accepts different parameters:
Common Parameters:
- `symbol`: Trading pair (e.g., BTCUSDT, ETHUSDT)
- `timeframe`: Candle timeframe (1m, 5m, 15m, 1h, 4h, 1d)
- `initial\_capital`: Starting capital (default: 10,000 USDT)
- `max\_position\_size`: Maximum position as percentage (default: 0.1)
Momentum-Specific:
- `period`: Lookback period for momentum calculation
- `threshold`: Minimum momentum to trigger entry
Mean Reversion-Specific:
- `period`: Period for Bollinger Bands
- `std\_dev`: Standard deviation multiplier
- `entry\_threshold`: Z-score for entry signal
Testing Strategies
Once created, test your strategy with realistic backtesting:
POST http://localhost:8788/api/strategies/{id}/backtest
{
 "start_date": "2024-01-01",
 "end_date": "2024-04-01",
 "initial_capital": 10000
}
Response includes:
- Total return (%)
- Sharpe ratio
- Maximum drawdown
- Win rate (%)
- Profit factor
- Number of trades
- Equity curve data

7. Discovery and Self-Evolution

SLATE has a powerful feature: its ability to autonomously discover and evolve trading strategies.
How Discovery Works
The discovery system follows this continuous loop:
1. **Generate**: Create diverse strategies using different types and parameters
2. **Test**: Run realistic backtests on historical data
3. **Evaluate**: Score strategies by risk-adjusted returns
4. **Select**: Keep top performers by strategy type
5. **Evolve**: Create optimized variants of best strategies
6. **Repeat**: Continue improving over time
Realistic Testing Assumptions
SLATE uses brutally honest testing to avoid overfitting:
- **Maker Fee**: 0.02% (when providing liquidity)
- **Taker Fee**: 0.05% (when taking liquidity)
- **Slippage**: 5 basis points (0.05%)
- **Fill Rate**: 95% (5% of orders don't fill)
- **Position Sizing**: Maximum 10% of capital per trade
Starting Discovery
Via Dashboard:
1. Navigate to Discovery Dashboard
2. Click "Start Discovery"
3. Set parameters (optional):
 - Number of cycles: 100
 - Workers: 3 (parallel testing)
 - Symbols: BTCUSDT, ETHUSDT
 - Timeframes: 15m, 1h, 4h
4. Click "Start"
Via API:
POST http://localhost:8788/api/realistic-discovery/start
{
 "cycles": 100,
 "workers": 3,
 "symbols": ["BTCUSDT", "ETHUSDT"],
 "timeframes": ["15m", "1h", "4h"]
}
Stigmergic Coordination
SLATE uses swarm intelligence to maintain strategy diversity:
- **Pheromone Trails**: Successful strategies leave "trails" for others to follow
- **Negative Feedback**: Over-explored areas become less attractive
- **Multi-Objective Optimization**: Balances return, risk, and novelty
- **Collective Learning**: All strategies share knowledge through the database
This ensures SLATE doesn't converge on a single strategy type but maintains a diverse portfolio.
Self-Evolution Process
Strategies evolve through these mechanisms:
1. **Selection**: Top 10% of strategies by Sharpe ratio survive
2. **Mutation**: Parameters are adjusted by $\pm 10\%$
3. **Crossover**: Successful features are combined
4. **Regeneration**: New strategies are created based on patterns

8. Step-by-Step Guide: How to use SLATE to discover profitable trading strategies:

Example 1: Run Comprehensive Discovery on BTC
Goal: Find the best performing strategy type for Bitcoin
Steps:
1. Start the SLATE server
2. Open Discovery Dashboard: http://localhost:8788/discovery-dashboard
3. Click "Start Discovery" with these settings:
 - Cycles: 200
 - Symbol: BTCUSDT
 - Timeframes: 15m, 1h, 4h
 - Strategy Types: All
4. Wait for completion (~30 minutes)
5. Sort results by "Total Return"
6. Select top 5 strategies for paper trading
What to look for:
- Strategies with >20% annual return
- Sharpe ratio >1.5
- Maximum drawdown <25%
- Consistent performance across timeframes
Example 2: Focus on USD Profit (Not Sharpe)
Goal: Find strategies that maximize absolute USD profits
Configuration:
POST http://localhost:8788/api/realistic-discovery/start
{
 "optimization_target": "total_return",
 "min_return": 0.15,
 "max_drawdown": 0.30,
 "cycles": 150,
 "symbols": ["BTCUSDT"],
 "timeframes": ["1h"]
}
Analysis:
After completion, query the database:
GET http://localhost:8788/api/realistic-discovery/results/top?limit=20&sort_by=total_return
Filter for:
- Total return >25%
- Profit factor >2.0
- Average win > average loss
- At least 50 trades for statistical significance
Example 3: Discover Multi-Timeframe Strategies
Goal: Find strategies that work across multiple timeframes
Process:
1. Run discovery on 15m timeframe (100 cycles)
2. Run discovery on 1h timeframe (100 cycles)
3. Run discovery on 4h timeframe (100 cycles)
4. Compare top performers from each
5. Look for strategy types that appear in all three
API Sequence:

Run on 15m

```
curl -X POST "http://localhost:8788/api/realistic-discovery/start?timeframe=15m&cycles=100"
```

Wait for completion, then run on 1h

```
curl -X POST "http://localhost:8788/api/realistic-discovery/start?timeframe=1h&cycles=100"
```

Wait for completion, then run on 4h

```
curl -X POST "http://localhost:8788/api/realistic-discovery/start?timeframe=4h&cycles=100"
```

****Identify Robust Strategies:****

- Same strategy type performs well on multiple timeframes
- Similar parameter values across timeframes
- Consistent risk-adjusted returns

Example 4: Low Volatility Regime Strategies

****Goal**:** Find strategies that work best during low volatility periods

****Setup**:**

```
POST http://localhost:8788/api/realistic-discovery/start
```

```
{
  "volatility_filter": "low",
  "atr_threshold": 0.02,
  "cycles": 100,
  "symbols": ["BTCUSDT", "ETHUSDT"],
  "timeframes": ["1h"]
}
```

****Expected Results**:**

- Mean reversion strategies should perform well
- Look for tight Bollinger Band parameters
- Short-term mean reversion (5-15 period)
- High win rate (>60%) acceptable

Example 5: High Volatility Breakout Strategies

****Goal**:** Find strategies that capture large moves

****Setup**:**

```
POST http://localhost:8788/api/realistic-discovery/start
```

```
{
  "volatility_filter": "high",
  "atr_multiplier": 2.0,
  "cycles": 100,
  "symbols": ["BTCUSDT"],
  "timeframes": ["15m", "1h"]
}
```

****Focus On**:**

- Breakout strategies
- Momentum strategies
- Wider stop losses to accommodate volatility
- Lower win rate acceptable if wins are large

Example 6: Portfolio Diversification Discovery

****Goal**:** Build a diverse portfolio of uncorrelated strategies

****Process**:**

1. Run discovery with 500 cycles
2. Get top 50 strategies by Sharpe ratio
3. Calculate correlation matrix
4. Select strategies with <0.3 correlation

****API Approach**:**

Get top strategies

```
curl "http://localhost:8788/api/realistic-discovery/results/top?limit=50" > top_strategies.json
```


Calculate correlation (this would be done in SLATE)

Select 5-10 uncorrelated strategies

```

**Ideal Portfolio:**
- Mix of strategy types (momentum, mean reversion, trend)
- Different timeframes
- Different entry/exit logic
- Low correlation to each other
### Example 7: Machine Learning Strategy Discovery
**Goal:** Let SLATE discover ML-based strategies
**Setup:**
POST http://localhost:8788/api/realistic-discovery/start
{
  "strategy_types": ["machine_learning"],
  "ml_features": ["returns", "rsi", "macd", "atr", "volume"],
  "cycles": 100,
  "symbols": ["BTCUSDT"],
  "timeframes": ["1h"]
}
**What to Expect:**
- Complex feature combinations
- Non-linear relationships
- May overfit - verify with walk-forward analysis
- Look for consistent out-of-sample performance
### Example 8: Regime-Switching Discovery
**Goal:** Find strategies that adapt to market conditions
**Setup:**
POST http://localhost:8788/api/realistic-discovery/start
{
  "strategy_types": ["regime_switching"],
  "regimes": ["trending", "ranging", "volatile"],
  "cycles": 150,
  "symbols": ["BTCUSDT", "ETHUSDT"],
  "timeframes": ["1h", "4h"]
}
**Evaluation:**
- Check performance in each regime separately
- Verify smooth regime transitions
- Look for strategies that avoid large drawdowns during regime changes
### Example 9: Parameter Optimization
**Goal:** Take a good strategy and optimize its parameters
**Process:**
1. Start with a known decent strategy (e.g., Moving Average Crossover)
2. Run focused discovery on that strategy type
3. Let SLATE vary parameters systematically
**Example:**
POST http://localhost:8788/api/realistic-discovery/start
{
  "base_strategy": "moving_average_crossover",
  "parameter_ranges": {
    "fast_period": [8, 12, 16, 20],
    "slow_period": [20, 24, 26, 30],
    "signal_period": [7, 9, 12]
  },
  "cycles": 200
}
**Analysis:**
- Plot parameter combinations vs returns
- Look for robust parameter regions (not just one optimal point)
- Avoid overfitting to specific values
### Example 10: Continuous Evolution Run
**Goal:** Let SLATE run overnight and discover strategies
**Setup:**
POST http://localhost:8788/api/realistic-discovery/start
{
  "cycles": 1000,
  "workers": 5,
  "symbols": ["BTCUSDT", "ETHUSDT", "SOLUSDT"],
  "timeframes": ["15m", "1h", "4h"],
  "auto_save": true,
  "save_interval": 50
}
**Next Morning:**
1. Check Discovery Dashboard
2. Export top strategies:
  curl "http://localhost:8788/api/realistic-discovery/export?format=json" > strategies.json
3. Review performance breakdown
4. Select best 5-10 strategies
5. Activate them for paper trading
### Interpreting Discovery Results
When analyzing discovery results, focus on:
**Primary Metrics:**
- **Total Return**: Actual USDT profit percentage
- **Sharpe Ratio**: Risk-adjusted return (>1.5 is good)
- **Maximum Drawdown**: Largest peak-to-trough decline (<30% is acceptable)
**Secondary Metrics:**
- **Win Rate**: Percentage of profitable trades
- **Profit Factor**: Gross wins / gross losses (>2.0 is good)
- **Calmar Ratio**: Return / max drawdown
**Red Flags:**
- Very high returns with very few trades (luck)
- Excellent backtest but poor walk-forward (overfitting)
- Extreme parameter values (unstable)
---

9. API Reference
SLATE provides a comprehensive REST API for all operations.
### Base URL
http://localhost:8788
### Strategy Management
#### Create Strategy
POST /api/strategies
Content-Type: application/json
{
  "name": "My Strategy",
  "type": "momentum",
  "symbol": "BTCUSDT",
  "timeframe": "1h",
  "parameters": {
    "period": 14
  }
}
#### List All Strategies
GET /api/strategies
#### Get Strategy Details
GET /api/strategies/{id}
#### Activate Strategy
POST /api/strategies/{id}/activate
#### Deactivate Strategy
POST /api/strategies/{id}/deactivate
#### Backtest Strategy
POST /api/strategies/{id}/backtest
{
  "start_date": "2024-01-01",
  "end_date": "2024-04-01",
  "initial_capital": 10000
}
### Discovery Operations
#### Start Discovery
POST /api/realistic-discovery/start
{
  "cycles": 100,
  "workers": 3,
  "symbols": ["BTCUSDT"],
  "timeframes": ["1h"]
}
#### Stop Discovery
POST /api/realistic-discovery/stop
#### Get Discovery Status
GET /api/realistic-discovery/status
#### Get Top Strategies
GET /api/realistic-discovery/results/top?limit=10&sort_by=total_return
#### Get Discovery Statistics
GET /api/realistic-discovery/statistics
### Cleanup Database
POST /api/realistic-discovery/cleanup
### Risk Management
#### Calculate Position Size
POST /api/risk/position-size
{
  "capital": 10000,
  "risk_percentage": 0.02,
  "stop_loss": 0.05
}
#### Kelly Criterion Calculation
POST /api/risk/kelly
{
  "win_rate": 0.55,
  "avg_win": 100,
  "avg_loss": 80
}
### Cross-Language Compilation
#### Python to HaasScript
POST /api/export/haas-script
{
  "code": "def strategy():\n return prices[0] > prices[1]"
}
#### HaasScript to Python
POST /api/import/haas-script
{
  "code": "price = IndexArray(prices, 1)"
}
#### Python to Pine Script
POST /api/export/pine-script
{
  "code": "def strategy():\n return close > open"
}
### Health & Monitoring
#### Health Check
GET /health
#### System Metrics
GET /api/metrics
#### Complete Health Summary
GET /api/health/summary
---

10. Troubleshooting
#### Issue 1: Port Already in Use
**Error:** `Address already in use`
**Solution:**
```

Find process using port 8788

Isof -ti:8788

Kill the process

kill -9 \$(ls -ti:8788)

Or use a different port


```
SLATE_PORT=8789 python3 -m slate_core.server
#### Issue 2: Module Import Errors
**Error:** `ModuleNotFoundError: No module named 'xxx'`
**Solution:**
```

Reinstall dependencies

```
pip install -r requirements.txt --force-reinstall
```

Or install specific module

pip install xxx

Issue 3: Discovery Not Starting

****Symptoms:**** Discovery stays in "starting" state

****Solution:****

1. Check if historical data exists:

 ls slate_core/palace_data/historical/

2. If empty, SLATE will fetch data automatically (may take a few minutes)

3. Check server logs for errors

Issue 4: Database Lock Errors

****Error:**** `database is locked`

****Solution:****

Remove lock files

```
rm -f slate_core/*.db-lock  
rm -f slate_core/palace_data/*.db-lock
```

Restart SLATE


```
python3 -m slate_core.server
#### Issue 5: Memory Issues
**Symptoms:** SLATE becomes slow or crashes
**Solution:**
1. Reduce number of discovery workers:
{
  "workers": 2 // instead of 5
}
2. Cleanup database:
  curl -X POST http://localhost:8788/api/realistic-discovery/cleanup
3. Reduce historical data retention
### Getting Help
If you encounter issues not covered here:
1. **Check the logs:** SLATE logs all activities
2. **Run tests:** `python3 slate_core/run_tests.py`
3. **Check GitHub Issues:** https://github.com/Tilanthi/SLATE/issues
4. **Review API docs:** http://localhost:8788/docs
### Best Practices
1. **Start Small:** Begin with short discovery runs (50-100 cycles)
2. **Monitor Regularly:** Check dashboards every few hours
3. **Clean Up Periodically:** Run database cleanup weekly
4. **Diversify:** Don't rely on a single strategy
5. **Validate:** Always verify results with walk-forward analysis
6. **Paper Trade First:** Never skip paper trading validation
7. **Keep Learning:** Review discovery logs to understand what works
---
```

Appendix

```
#### Glossary
- **Alpha:** Returns above the market benchmark
- **Drawdown:** Peak-to-trough decline in value
- **Kelly Criterion:** Formula for optimal position sizing
- **Sharpe Ratio:** Risk-adjusted return measure
- **Slippage:** Difference between expected and actual execution price
- **Stigmergy:** Indirect coordination through environment modification
- **VaR (Value at Risk):** Maximum expected loss over a time period
### Quick Reference Cards
**Ports:**
- Main Server: 8788
- Dashboard: http://localhost:8788/dashboard
- Discovery: http://localhost:8788/discovery-dashboard
- API Docs: http://localhost:8788/docs
**Commands:**
```

Start SLATE

```
python3 -m slate_core.server
```

Run tests

```
python3 slate_core/run_tests.py
```

Start discovery

```
curl -X POST http://localhost:8788/api/discovery/trigger
```

Get top strategies


```
curl http://localhost:8788/api/realistic-discovery/results/top
```

```
**Default Parameters:**
```

- Initial Capital: 10,000 USDT
- Max Position Size: 10% of capital
- Maker Fee: 0.02%
- Taker Fee: 0.05%
- Slippage: 5 bps (0.05%)

```
---
```

```
**Version:** 1.0.0
```

```
**Last Updated:** April 30, 2026
```

```
**Mode:** PAPER_TRADING ONLY
```

```
**⚠ IMPORTANT:** SLATE is for paper trading and simulation only. Never use SLATE for live trading without proper testing and risk management.
```

```
---
```

```
*For the latest updates and documentation, visit https://github.com/Tilanthi/SLATE*
```

Quick Reference

□ Server Port	8788
□ Main Dashboard	http://localhost:8788/dashboard
□ Discovery Dashboard	http://localhost:8788/discovery-dashboard
□ API Documentation	http://localhost:8788/docs
⚠ Mode	Paper Trading Only
□ Initial Capital	10,000 USDT
□ Maker Fee	0.02%
□ Taker Fee	0.05%
□ Slippage	5 bps (0.05%)
□ Max Position	10% of capital